

FIR Filter Implemented with the Self-purging Redundancy Technique

DOCUMENTATION

June 2026

1 Introduction

A FIR filter (Finite Impulse Response filter) is a filter whose output depends only on a finite number of current and past input samples. It is described by:

$$y[n] = \sum_{k=0}^{N-1} h[k] x[n-k]$$

where:

- $x[n]$ is the input signal,
- $y[n]$ is the output signal,
- $h[k]$ are the filter coefficients (taps)

Each input sample is multiplied by a coefficient and the products are summed together. In hardware, FIR filters are typically implemented using a chain of multipliers, adders, and delay registers.

Self-Purging Redundancy (SPR) is a fault-tolerance technique that employs multiple redundant instances of a module and automatically excludes faulty instances from the final decision process.

For a self-purging redundancy (SPR) system with N redundant modules and majority-based fault detection the system can tolerate up to $\lfloor \frac{N-1}{2} \rfloor$ faulty modules (or faulty inputs to the voter) while still producing the correct output. This is the same limit as a conventional N -modular redundancy (NMR) system because the correct modules must remain in the majority. The advantage of self-purging redundancy is that once a module is identified as faulty, it is disabled (purged) from future voting.

The operation of SPR can be summarized as follows:

1. Replicate the module M times.

2. Apply the same input to all replicas.
3. Compare the outputs of all replicas.
4. Identify modules whose outputs agree with the majority.
5. Exclude modules whose outputs deviate beyond a predefined tolerance threshold.
6. Generate the final output using only the trusted replicas.

Here we apply SPR on the FIR filter at the MAC module level - each "tap" has M redundant MAC modules whose outputs are fed to a voter.

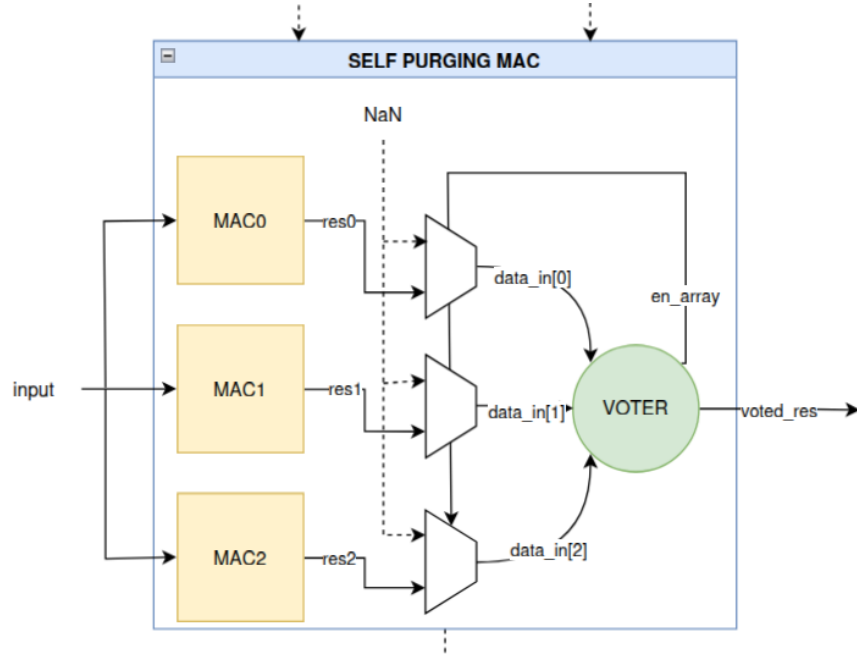


Figure 1: Example of the SPR technique applied to a MAC module

2 Architecture

The designed Self-purging FIR filter was written to be fully parametrized. This means that the input data width, filter order and number of redundant MAC modules can be changed through the constants set in the top module of the project.

Since both the MAC modules and the voter have several pipeline stages, the multiplication inputs for each successive Self-purging MAC module within the

filter need to be delayed by the number of pipeline phases present in both the MAC modules and the voter (Figure 2).

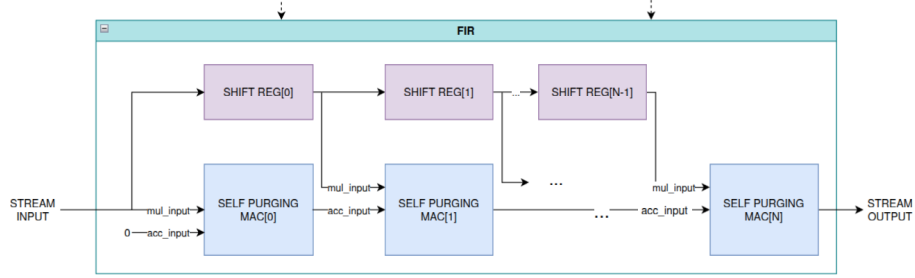


Figure 2: FIR Filter - Modular Design

The implementation of the shift registers is trivial and what we can note is that we use an enable signal to stall the whole filter when the data input is not valid. This signal is visible on Figure 3 for the shift registers as well.

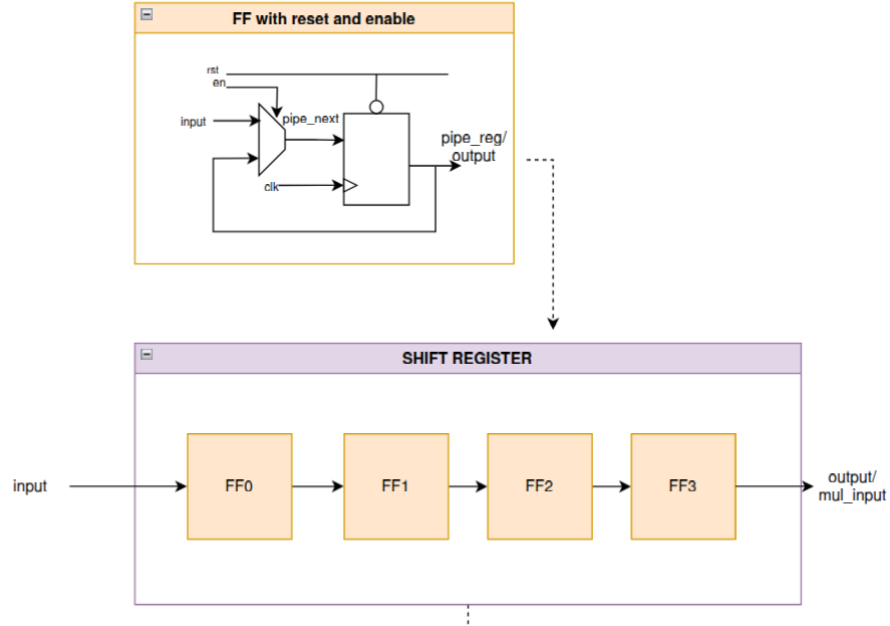


Figure 3: Shift register implementation

The MAC modules are straightforward, multiply and accumulate operation with a pipeline installed to improve throughput (Figure 4). The outputs are fed to a voter (Figure 5) which compares each input with each other using XOR

gates. The tolerance of the comparison can be adjusted by changing the number of bits that are compared. This comparison along with the current values of the inputs determines the switch enable signals and the voted output.

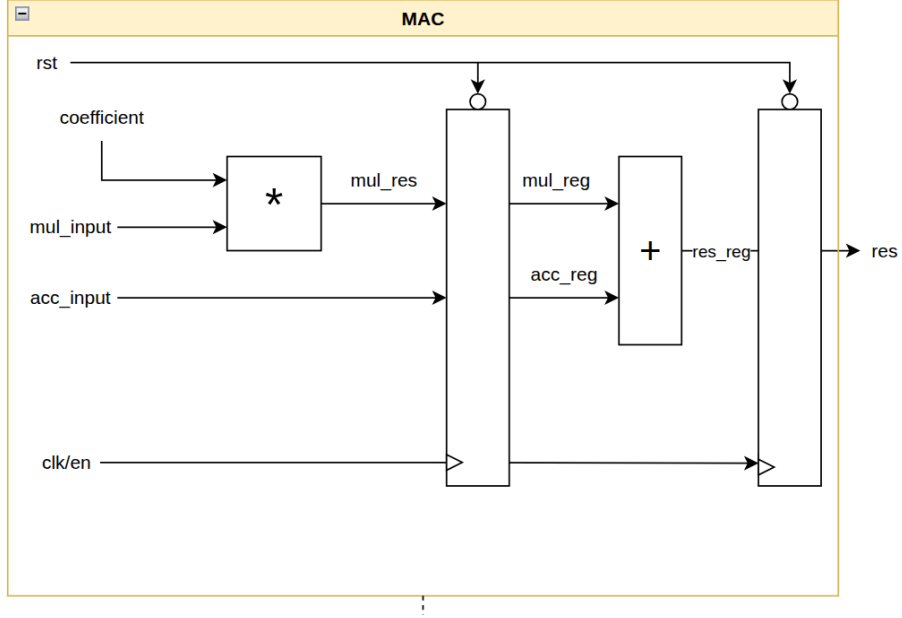


Figure 4: MAC Design

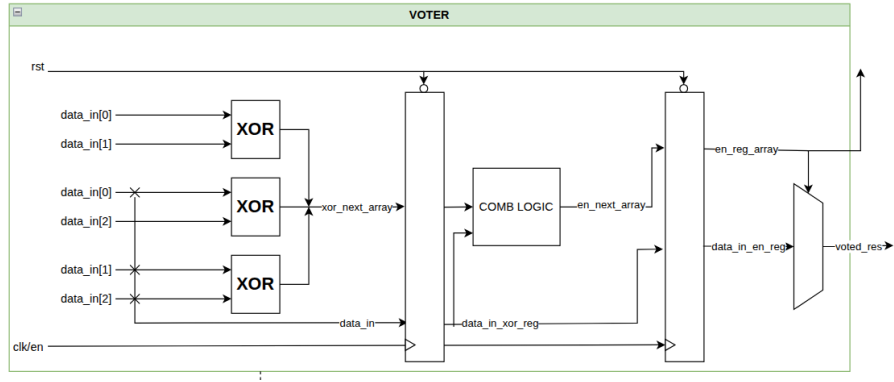


Figure 5: Voter Design

The "*COMB LOGIC*" block in the voter is responsible for generating the enable signal for each input module. If a module has not already been disabled, the next enable state, denoted as *en_next_array* in Figure 5, is computed using

the outputs of XOR comparisons between the given input and all other currently active (non-purged) inputs.

Each XOR comparison result is denoted as y_{ij} , where $y_{ij} = 1$ indicates a mismatch between inputs i and j , and $y_{ij} = 0$ indicates agreement. These values are then used to determine whether a module should remain active or be disabled. Specifically, a module is disabled if it disagrees with all other active inputs:

$$en_k = \begin{cases} \bigvee \bar{y}_{ij}, & \text{for } i = k \vee j = k \end{cases}$$

3 Testing results

Running a simulation of the filter for three redundant modules and three "taps" or three coefficients gives us the waveforms shown in Figure 6. The green colored signals are the inputs and outputs of the filter while the yellow, blue, and pink colored signals are, respectively, the inputs and outputs of Self-purging MAC modules. We can clearly see that for each MAC (or for each of the colors) the results are shifted by the number of pipelines we have per MAC module.

If we were to look at the first valid input into the filter we can see that it's corresponding output appears twelve clocks later. This number corresponds to the product of the number of "taps" and the number of pipelines that we have within one self-purging MAC module.

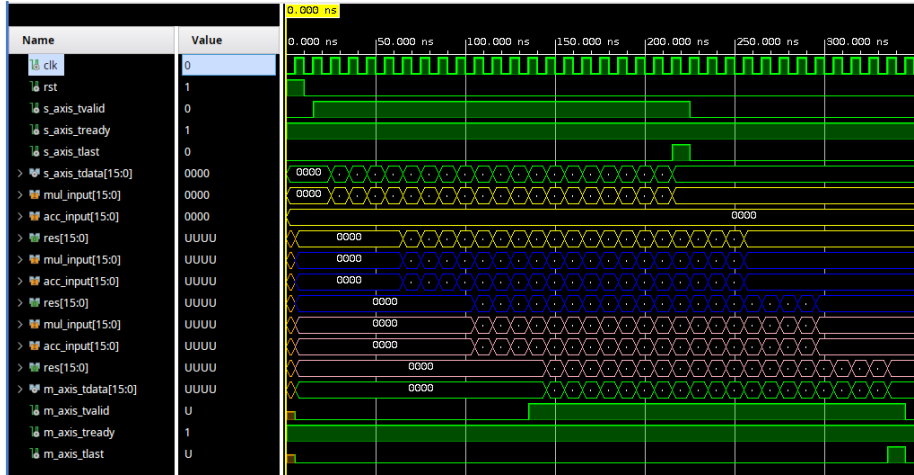


Figure 6: Simulation of filter

To simulate a stuck-at fault in the system we use tcl commands. For the

purpose of including errors in this project we used the following sequence:

```
restart
add_force {'signal_path'} -radix hex {0000 50ns}
run 200ns
```

While to undo the effect of the set stuck-at fault previously applied we use the tcl command:

```
remove_force {'signal_path'}
```

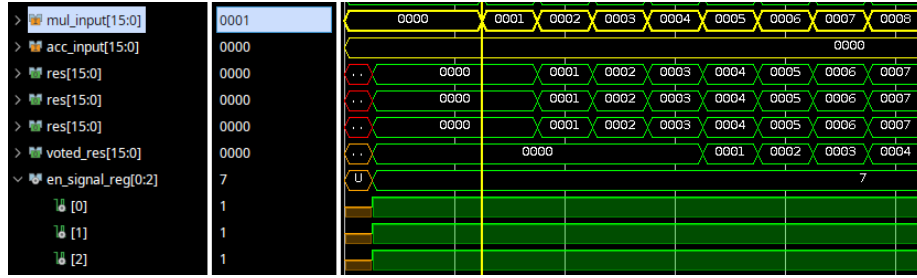


Figure 7: Voted result and switch enable signals when there is no fault present

We expect to see the same output chosen when there is no fault (Figure 7) as well as when there is a single stuck-at fault (Figure 8). The difference in those two cases would be the switch enable signal, which should disable the MAC with the stuck-at fault.

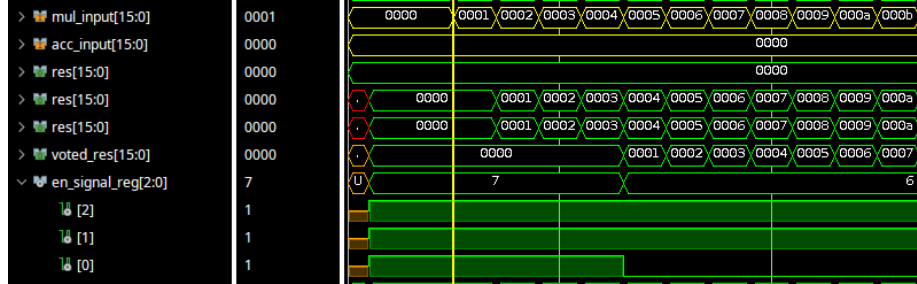


Figure 8: Voted result and switch enable signals when there is a single stuck-at fault present

Now, the voting is done using only the two remaining modules. If their outputs should disagree, both would be flagged as faulty by the enable signal since we have no way of knowing whether either of the two outputs is correct. The output fed to the next MAC module would be chosen arbitrarily by its index number - the MAC with the lower index module will become the default output until the system is fixed (Figure 9).

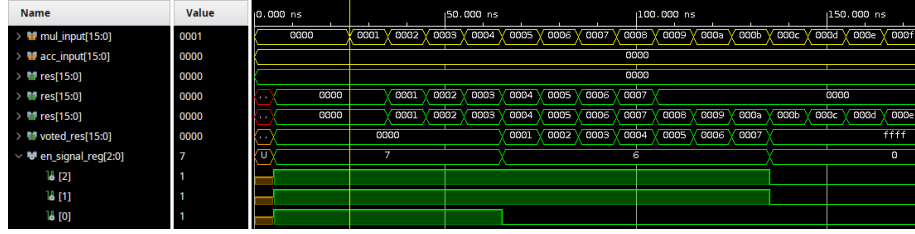


Figure 9: Voted result and switch enable signals when there are two cascading stuck-at faults present

It should be noted that if the two stuck-at faults had the same value and they happened at the same time, then the voter would pick their result as the correct one and exclude the third output (Figure 10). The chances of such an event are low, but still possible.

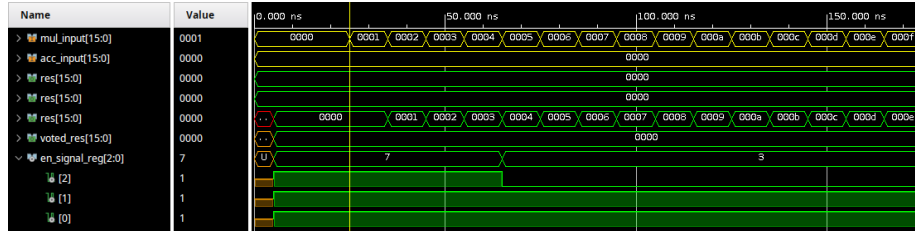


Figure 10: Voted result and switch enable signals when there are two simultaneous stuck-at faults present

4 Timing and resource analysis

To evaluate how varying the number of redundant MAC modules feeding the voter affects frequency and resource utilization, we first define a timing constraint. In this project, the chosen maximum clock period is 10ns, matching the one used in the testbench. Then we set the desired number of redundant modules by changing the value of the constant *numSelfPurgingMAC* in the top module of the design before running the synthesis and implementation processes. The synthesis reports resource utilization while implementation reports the timing.

We expect the design frequency to decrease as the number of redundant modules increases due to the added complexity of the voter logic. As more redundant modules are included, the voter must handle a larger number of inputs, which increases its fan-in and makes the combinational logic more complex. Since the maximum operating frequency is determined by the longest (critical) delay path in the design, which is a path located in the voter, any increase in voter complexity increases the critical path delay and therefore reduces the achievable

frequency. The relationship between the maximum achievable frequency and number of used redundant modules is shown in the table below.

Table 1: Achievable frequency and corresponding period for a given number of redundant modules

Modules	Period [ns]	Frequency [MHz]
3	3.885	257.40
5	4.723	211.73
7	4.867	205.46

For resource utilization we expect an increase in resources used for the synthesis of the MAC modules and voter logic. This relationship is shown in table 2 below.

Table 2: Detailed resource utilization for different numbers of redundant modules

Modules	FDRE	LUTs (total)	CARRY4	SRL16E	IO (IBUF+OBUF)
3	473	377	26	34	40
5	701	732	68	34	40
7	911	1199	134	34	40

The number of FDREs grows with increasing redundancy, which reflects the additional state registers required to manage the extra modules and support the self-purging mechanism. Similarly, the LUT usage increases significantly, as this corresponds to the growing complexity of the voter logic and the increased number of comparisons required between redundant inputs. The number of CARRY4 primitives also rises, indicating that the arithmetic and comparison structures within the design become more complex as more modules are added. In contrast, the usage of SRL16E primitives remains constant across all configurations, showing that the memory-related components of the design are unaffected by redundancy scaling. Likewise, the number of input/output buffers (IBUF and OBUF) does not change, confirming that the external interface of the system remains fixed regardless of the number of redundant modules.